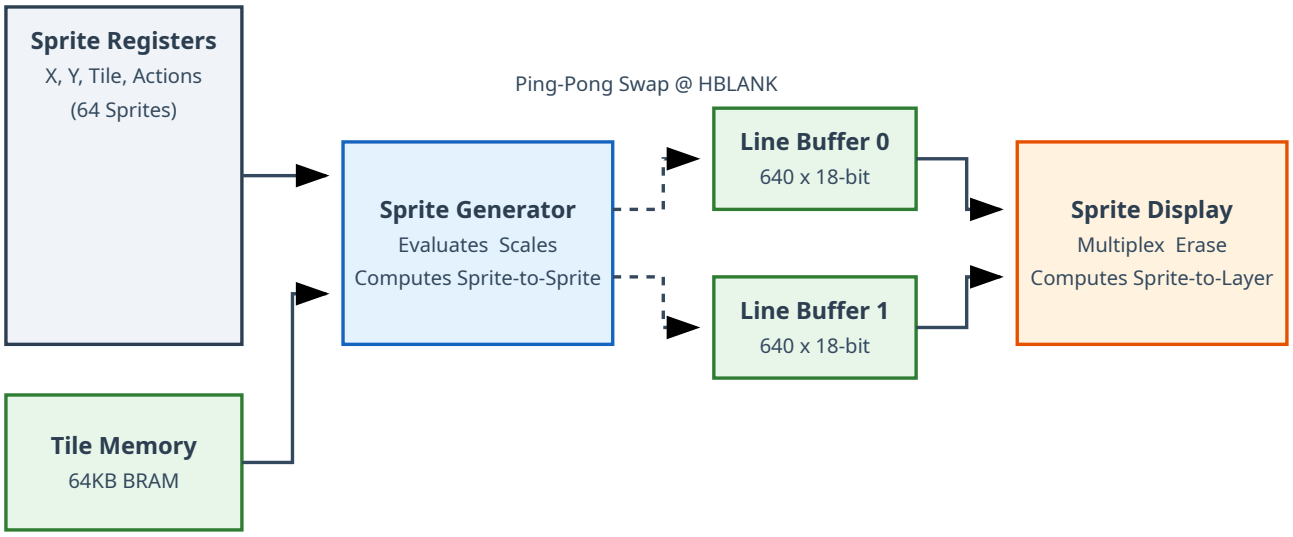
PAWSv3 Sprite Engine Documentation

1. Hardware Architecture Overview

The PAWSv3 sprite engine utilizes a split rendering architecture designed to decouple the sprite generation logic from the VGA display beam. This is achieved using a dual-unit system connected via ping-pong line buffers:

- **Sprite Generator Unit:** Operates independently of the display beam, processing the 64 internal sprite registers and drawing visible pixels into an off-screen line buffer.
- **Sprite Display Unit:** Synchronized strictly to the video clock. It reads the fully composed scanline from the active line buffer and multiplexes it with the background, bitmap, and tilemap layers.
- **Ping-Pong Line Buffers:** Two 640-pixel, 18-bit dual-port BRAMs. While the Display Unit streams Line Buffer 0 to the screen and automatically erases it (tail-clearing), the Generator Unit actively draws the next scanline into Line Buffer 1. The buffers swap every horizontal blanking (HBLANK) period.



2. Sprite Tile Memory

The engine utilizes a single, consolidated 65536-byte (64KB) True Dual-Port Block RAM (BRAM) to store sprite graphics.

- **Capacity:** Accommodates 256 individual 16x16 pixel sprite tiles.
- **Format:** Each pixel is an 8-bit index mapping to the PAWS 256-color palette. The value 64 (0x40) is designated as transparent.
- **Write Access:** The CPU accesses the memory sequentially via DMA using SPRITE_WRITER_NUMBER (to set the 8-bit tile index) and SPRITE_WRITER_COLOUR (to write successive pixels).

3. Sprite Actions & Bitfield

Each of the 64 sprites has a dedicated 9-bit SPRITE_ACTIONS register (uint9 actions) controlling transformations, scale, and layer depth.

Bits [8:7]	Bit [6]	Bit [5]	Bits [4:3]	Bits [2:0]
Layer ID (0-3)	32x32 Mode	Scale Dir.	Shift Amount	Rotation Reflection

Action Bit Breakdown

- **Bits [2:0] (Rotation & Reflection):** Applies coordinate transforms before memory fetch.
- 1: Reflect X (REFLECT_X).
- 2: Reflect Y (REFLECT_Y).
- 4: Rotate 0° (ROTATE0).
- 5: Rotate 90° (ROTATE90).
- 6: Rotate 180° (ROTATE180).
- 7: Rotate 270° (ROTATE270).
- **Bits [4:3] (Blocky Scale Factor):** Dictates the bit-shift multiplier for dynamic scaling.
- 0: 1x Scale.
- 1: 2x or 0.5x Scale (SPRITE_DOUBLE).
- 2: 4x or 0.25x Scale (SPRITE_QUAD).
- *Note: When scaling up, the hardware duplicates pixels, rendering the original tile graphics as "blocky" representations.*.
- **Bit [5] (Scale Direction):** Modifies the behavior of the Scale Factor.
- 0: Grow (Bitwise Left Shift <<).
- 1: Shrink (Bitwise Right Shift >>>) (SPRITE_HALF, SPRITE_QUARTER).
- **Bit [6] (32x32 Composite Mode):** When set (SPRITE_32X32), the hardware natively configures the sprite as a 32x32 pixel block. The engine reads 4 contiguous 16x16 tiles from the Sprite Memory (combining them into a larger grid) rather than scaling a single tile.

- **Bits [8:7] (Display Layer):** Dictates the rendering priority (SPRITE_LAYER_0 through SPRITE_LAYER_3). The Generator embeds this layer ID directly into the 18-bit Line Buffer data for the Display Unit to interpret.

4. Collision Detection System

The engine features a dual-tier, pixel-perfect collision detection system tracked automatically by the hardware.

1. **Sprite-to-Sprite Collisions (Generator Unit):** During the DRAW_WRITE phase, the generator reads the line buffer before plotting a new pixel. If an opaque pixel from another sprite already exists at that coordinate, a collision is flagged. Both the drawing sprite and the existing sprite have their respective bits set in a 64-bit detect_collision matrix.
2. **Sprite-to-Layer Collisions (Display Unit):** During the DISPLAY OUT phase, as pixels are streamed to the screen, the sprite's opacity is compared synchronously against the multiplexed visibility of the background, bitmaps (0/1), tilemaps (2-5), and character map (6). Results are compiled into a 7-bit detect_layer mask.

   Both collision accumulators are dumped into the CPU-readable Dual-Port BRAMs (sprite_collision_memory and sprite_layer_collision_memory) exclusively during the VBLANK/End-of-Frame period, ensuring stable reads from software.

5. PAWSlibrary C API Functions

The PAWSlibrary provides several abstractions for interacting with the sprite hardware.

Memory Management

- void set_sprite_bitmaps(unsigned char tile_number, unsigned char *sprite_bitmaps, unsigned int bytes) Writes a raw pixel array to the sprite BRAM starting at tile_number via DMA.
- void set_sprite_bitamps_from_spritesheet(unsigned char tile_number, unsigned char count, unsigned char *sprite_bitmaps, unsigned char mode) Extracts and uploads either 16x16 or 32x32 blocks from a unified spritesheet.
- void set_sprite_bitamps_from_spritesheet32x32(unsigned char tile_number, unsigned char *sprite_bitmaps) Extracts 4 separate 16x16 tiles from a spritesheet to form a composite 32x32 sprite block.

Sprite Manipulation

- void set_sprite(unsigned char sprite_number, unsigned char active, short x, short y, unsigned char tile, unsigned short sprite_attributes) Configures all hardware registers for a specific sprite. It automatically calculates the physical bounding-box size for the hardware loops by inspecting the SPRITE_32X32 and scaling bits.
- void set_sprite32(unsigned char sprite_number, unsigned char active, short x, short y, unsigned char tile, unsigned char sprite_actions) A helper function that orchestrates four independent 16x16 hardware sprites (arranged in a 2x2 grid) to act as a single logical 32x32 entity, automatically handling rotations and reflections across the quadrant.
- void update_sprite(unsigned char sprite_number, unsigned char kill, short dx, short dy, unsigned char dt) Adjusts a sprite's X/Y coordinates and animation frame (dt), automatically handling screen wrapping and bounds-checking.

Getters and Setters

- void set_sprite_attribute(unsigned char sprite_number, unsigned char attribute, short value) Modifies an individual property (e.g., ATTR_SPRITE_X, ATTR_SPRITE_TILE).
- short get_sprite_attribute(unsigned char sprite_number, unsigned char attribute) Retrieves the current value of a sprite's property.

- void set_sprite_layer(unsigned char sprite_number, unsigned char sprite_layer) Safely manipulates bits 7 and 8 of the action register to move a sprite between rendering layers.

Collision Readout

- unsigned long get_sprite_collision(unsigned char sprite_number) Returns the 64-bit mask indicating which other sprites this sprite touched during the last frame.
- unsigned short get_sprite_layer_collision(unsigned char sprite_number) Returns a 7-bit mask indicating collision with environmental layers (bitmaps, tilemaps, etc.).